

# CaseWhy — USCIS Torch Demo Script (target: ~21 minutes inside the 30-minute Teams slot)

1. Real format, confirmed against `developer.uscis.gov`: a 30-minute Microsoft Teams call, Wednesday or Thursday 1:00–2:00 PM ET, recorded, audio required.
2. Peter attends alone — USCIS explicitly allows the primary business contact and the technical demo lead to be the same person.
3. Results arrive by email within 2 business days; a fail routes to a 15-minute "Re-Demo Office Hours" on just the missed criteria.
4. Shows USCIS's five real Case Status API criteria, in order:
  1. Data entry usability.
  2. JSON payload conversion.
  3. OAuth 2.0.
  4. HTTPS response handling — success and error, message shown in the UI.
  5. Case status tracking.
5. **File upload is a FOIA API criterion, not Case Status — deliberately not in this script.**
6. **Round 130** — the actual USCIS API Demo Checklist (sent alongside the scheduling link, separate from and broader than the five numbered criteria above) adds real, distinct requirements this script now also covers:
  1. The browser's own DevTools Network tab specifically (Step 2).
  2. CaseWhy's own frontend/backend authentication as its own point, separate from USCIS OAuth (Step 1).
  3. Manually running the scheduled batch job live, showing both a success and an error response in the same run (new Step 6).

---

## Before the call

---

1. **Confirm the `demo_id` step is already done and accepted by USCIS.**

1. USCIS won't send the scheduler link until they've seen proof `demo_id` is wired in — that has to happen before this call can even be booked.
2. See "demo\_id setup," below.
2. **Set up a dedicated demo account with no real PII.**
  1. A fresh sign-up, then upgrade it to Plus via `/plus` → Stripe Checkout using a Stripe test card ( `4242 4242 4242 4242` , any future expiry/CVC).
  2. Billing is in test mode, so this is a real checkout with no real charge.
  3. Plus is needed so the on-demand check / request-preview / error-detail features are all available live.
  4. Track cases only under confirmed sandbox test receipt numbers (table at the bottom of this script) — including `EAC111111111` specifically, the deliberate error case for Step 6.
  5. Do not use Peter's own real account or any account with a genuine tracked case.
3. **Have `docs/uscis/technical-brief.md` (PDF) and `docs/uscis/demo-qa.md` open or printed.**
  1. **This is the entire fallback plan, and the only one.**
  2. Round 130 — no AI tool of any kind is part of this demo, in the meeting or anywhere else on the machine during it: not a live chat window, not a background assistant, nothing.
  3. If a question stumps you, pause, and read the answer from these two documents.
  4. That's the plan; there isn't a second one.
4. **Confirm the sandbox is up before the call.**
  1. It operates 7 AM–8 PM ET only.
  2. This call is scheduled 1–2 PM ET, inside that window, but re-check the same morning.
5. **Rehearse the whole script once against the sandbox, with Code watching the logs, before the `demo_id` screenshot is even sent.**
  1. Not done as of this document being written — do this first.
6. **Check `app.casewhy.com` loads normally right before the call** (`curl -I https://app.casewhy.com` , or just open it).

1. Vercel's automatic bot/DDoS mitigation has tripped on this domain before from a burst of rapid API calls — exactly what rehearsing this script, or the call itself, looks like.
2. If the site 403s with a "Security Checkpoint" page, run `vercel firewall system-mitigations pause` from a real terminal inside the `casewhy` project directory (it needs an interactive confirmation, so a Cloud session can't run it for you) and re-check.
7. **Export `ADMIN_DIAG_SECRET` in the terminal you'll use for Step 6 below, before the call starts** (`export ADMIN_DIAG_SECRET=<value from Vercel's production environment>`).
  1. The live batch-job command references it as `$ADMIN_DIAG_SECRET` so nothing needs typing or pasting live.
  2. Do this before screen sharing begins, not during.
8. **Track `EAC111111111` on the demo account ahead of time**, alongside the normal working cases.
  1. It's a real, well-formed, permanently-unrecognized receipt number (confirmed live: always a real USCIS 404, "Case Status Online does not recognize the receipt number entered").
  2. This is the deliberate, repeatable error case for Step 6.

## **demo\_id setup (must happen before the call can be scheduled)**

---

1. USCIS assigns a 4-digit ID. Set it as `USCIS_DEMO_ID` in Vercel's production environment (`vercel env add USCIS_DEMO_ID`).
2. **Redeploy production** (`vercel --prod`) — a new/changed environment variable isn't picked up by already-running functions until the next deploy.
3. Confirm it's live: use the on-demand check button on any tracked case (Step 2 below) and open "Show technical details" — the `demo_id` header should appear in the redacted request preview. That screenshot *is* the proof USCIS wants.
4. Email that screenshot to Torch Developer Support. The scheduler link comes back after they see it.
5. **Remove `USCIS_DEMO_ID` from Vercel and redeploy again after the real demo is done** — it should only be set during the actual demo

window, not permanently.

---

## Script

---

### Step 1 — Sign in, show the interface (2 min)

#### Click:

1. Navigate to `app.casewhy.com`.
2. Sign in to the dedicated demo account (or show the sign-in flow live).

#### Say:

1. "This is CaseWhy, a case-status tracker. I'll walk through all five criteria: data entry, the JSON payload, OAuth 2.0, error handling, and case tracking."
2. "One thing worth being explicit about here: this sign-in is our own app's authentication — a session with our own server, through Neon Auth — separate from the OAuth 2.0 flow our server itself uses to talk to USCIS, which I'll show in a minute."
3. "Two different authentication relationships: the browser to us, and us to USCIS."

#### Reviewer sees:

1. The dashboard.
2. The receipt-number field's live format feedback (type a few characters, show the red/green indicator).
3. The Spanish toggle.
4. Resize the window briefly to show mobile responsiveness.

**Criterion:** #1, data entry usability, plus the checklist's separate "authentication between the frontend and backend of your application" line.

#### If it fails live:

1. If sign-in hangs, refresh once (a known-clean path from rehearsal).
2. If the sandbox itself is down: "Here's a recording of this exact flow from rehearsal," and switch to it.

## Step 2 — Look up and track a case, show the JSON payload (4 min)

### Click:

1. Open the browser's DevTools Network tab before submitting anything, and leave it visible.
2. Enter `WAC9999999999` (I-751, confirmed sandbox test number) in the receipt field → click "Look up status."
3. Point at the request that appears in the Network tab (a request to `app.casewhy.com`, our own server).
4. Then look at the result card's "Track this case" button right under the status line. Click it (case type defaults to "Other," one click saves).
5. Then click "Check now," then "Show technical details."

### Say:

1. "Before I go further — the DevTools Network tab, since that's specifically asked for."
2. "You'll see the request from my browser to our own server here, but you will not see a request to USCIS's API directly in this tab, and that's deliberate, not a gap: our OAuth credentials and the USCIS call itself run entirely server-side, so they never reach the browser at all."
3. "What I can show you instead is the exact outbound request our server builds — same shape, same headers, generated by the identical function that makes the real call, not a separate description — right here in 'Show technical details.'"
4. "Looking up a case is a real live call either way, but it doesn't save anything by itself — that's this separate 'Track this case' button, right here where you can't miss it."

### Reviewer sees:

1. The browser-to-CaseWhy request in DevTools' Network tab.
2. The new case appear in the always-visible tracked-cases list on the left.
3. The outbound-to-USCIS request panel (method, URL, headers, including the `demo_id` header from the setup step above).

**Criterion:** #1 (a real look-up-vs-save distinction, not a confusing overload), #2 (JSON payload conversion), and the checklist's Network-tab requirement.

**If it fails live:** if `WAC9999999999` doesn't return a result (sandbox data can shift), use `WAC0000000001` as backup — both are on CaseWhy's own confirmed-working list.

### Step 3 — OAuth 2.0 (2 min)

**Say, over the same screen:**

1. "Authentication is OAuth 2.0 Client Credentials — token fetched and cached server-side, refreshed automatically before expiry, and it never reaches the browser."
2. "I can show the client code directly if useful."
3. (Have `src/lib/uscis/client.ts` open in a second tab, ready to share if asked.)

**Reviewer sees:** the code, if requested; otherwise this is narrated over Step 2's result.

**Criterion:** #3, OAuth 2.0.

**If it fails live:** no live dependency — this step can't meaningfully fail.

### Step 4 — Error handling, success and failure, both visible (4 min)

**Click:**

1. Track a malformed number first: type `AB12345` in the field (do not submit — just show the client-side feedback), then correct it.
2. Then, in a **separate** case-add attempt, use `EAC0000099999` (well-formed, not a real case) and let it actually submit and fail.

**Say:** "USCIS's own API returns a real 422 for a malformed number and a 404 for one it doesn't recognize — both show up as USCIS's actual message here, not a generic error."

**Reviewer sees:**

1. The client-side red/green format feedback for the malformed number.
2. Then a real, specific USCIS error message on-screen for the unknown one.

**Criterion:** #4, HTTPS response handling (both success — Step 2 — and error, this step).

**If it fails live:** if `EAC0000099999` unexpectedly returns a real hit (a sandbox pool quirk), say so honestly and try a different well-formed-but-unlikely number rather than force it.

## **Step 5 — Case status tracking end to end (4 min)**

### **Click:**

1. Point at the tracked-cases list itself — every case shown with its receipt, type, last known status, and last-checked time, straight from the database.
2. Click the pre-tracked case ( `EAC9999103403`, I-130, "Case Was Approved") to open its full detail — status text and history.
3. If it has more than one history entry, open the history view.
4. Then, back in the list, click "Stop tracking" on the case added in Step 2, then "Yes" to confirm.

### **Say:**

1. "The list itself is always populated from our database, not just a live call — even if a refresh fails, you still see every tracked case with its last known status, not a blank page."
2. "This case is checked on a schedule; a genuine status change triggers an email and, if enabled, a push notification — never on the very first check."
3. "Removing a case is self-serve and immediate, with a confirm step."

### **Reviewer sees:**

1. The always-visible list.
2. The status/history display.
3. The case disappearing from the list immediately after confirming removal.

**Criterion:** #5, case status tracking.

**If it fails live:** if the pre-tracked case shows only one history entry, describe the notification mechanism directly rather than forcing an empty view to look busy.

## **Step 6 — Batch job, run live (3 min)**

### **Before this step:**

1. `ADMIN_DIAG_SECRET` already exported in this terminal (prep step 7, above).
2. `EAC1111111111` already tracked on the demo account (prep step 8, above) alongside the normal working cases.
3. Terminal window visible on the shared screen, large enough to read.

**Click:** Run this exact command, live:

```
curl -X POST https://app.casewhy.com/api/cron/check-status \
-H "Authorization: Bearer $ADMIN_DIAG_SECRET"
```

**Say, before running it:**

1. "This is the scheduled job that checks every tracked case — it runs automatically once a day, every day."
2. "I'm triggering it by hand right now, outside its normal schedule, so you can see it live rather than take my word for it."
3. "It's the same authenticated call our external scheduler makes, just run manually."

**Say, reading the JSON response:**

1. "`checked` is how many went through cleanly — that's the success case."
2. "`errors` is where a per-case failure shows up without taking down the rest of the run."
3. "That entry right there is `EAC1111111111`, a receipt number I deliberately tracked because it's not a real case, so it reliably 404s every time."
4. "That's not a bug, it's the error path working exactly as designed, and it's isolated: the other case still gets checked and counted in `checked`, not swallowed by the one that failed."

**Reviewer sees:** the raw JSON response in the terminal — `checked`, `notified`, `errors` (containing the real USCIS 404 for `EAC1111111111`), `stoppedEarly`, `remaining`.

**Criterion:** the checklist's batch-update requirement — manually run, live, success and error both shown in the same run.

**If it fails live:**

1. If the response's `errors` array is empty (the demo account's tracked cases somehow don't include `EAC1111111111` — a setup miss, not a live failure), say so plainly.
2. Re-add the case via the dashboard on screen, then re-run the command once tracked.

## Step 7 — Close (2 min)

**Say:** "Everything I've shown is in the technical brief we sent ahead of this call, with the exact file and function for every claim. Happy to go deeper on anything, live or in a follow-up."

---

## Sandbox test numbers used in this script

| Receipt number             | Form  | Status                                                       | Used for                                   |
|----------------------------|-------|--------------------------------------------------------------|--------------------------------------------|
| <code>EAC9999103403</code> | I-130 | Case Was Approved                                            | Pre-tracked, Step 5                        |
| <code>WAC9999999999</code> | I-751 | Case Was Received At My Local Office                         | Added live, Step 2                         |
| <code>WAC0000000001</code> | I-751 | Case Was Transferred                                         | Backup for Step 2                          |
| <code>EAC0000099999</code> | —     | Deliberately unknown (real 404)                              | Step 4                                     |
| <code>EAC1111111111</code> | —     | Deliberately unknown (real 404, verified live round 130)     | Pre-tracked, Step 6's batch-job error case |
| <code>AB12345</code>       | —     | Deliberately malformed (real 422, client-side feedback only) | Step 4                                     |

1. All six are real sandbox behavior, confirmed live — either from CaseWhy's own confirmed-working pool, or a deliberately unrecognized/malformed number chosen specifically to reproduce a real USCIS error on demand.
2. No real PII, no real cases, on the whole call.

## **Real-time fallback rule, for anything not covered above**

---

1. If something fails live that isn't covered by a specific fallback: say so plainly ("that's not behaving as expected right now — I'll confirm and follow up").
2. Don't improvise an unverified claim, and move on.
3. Honest and brief beats a guess in front of a federal reviewer.